

AGENTTRADE
A BULL-BEAR DEBATE DRIVEN MULTI-AGENT FRAMEWORK FOR
STOCK ANALYSIS AND RISK-CONTROLLED PAPER TRADING

PROJECT REPORT FOR THE COURSE
AD23731 FOUNDATIONS OF AGENTIC AI

Submitted by

DONEESWARAN J. (Reg. No. 231501502)
NISHA S. (Reg. No. 231501112)



DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND
MACHINE LEARNING

RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, CHENNAI, 602105

OCTOBER 2026



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this is a Bonafide Record of the project work done by **Doneeswaran J. (231501502)** and **Nisha S. (231501112)** of VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in charge

Submitted for the Project Review held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

S. NO.	TITLE	PAGE NO.
1.	Introduction	1
2.	Problem Statement and Business case	2
3.	User and Stakeholder Analysis	4
4.	Proposed Agentic AI Solution	6
5.	Dataset and Knowledge Sources	9
6.	Tools and Technologies Used	11
7.	Agent Architecture	13
8.	Agent Workflow and Orchestration	16
9.	Agentic RAG Implementation	18
10.	System Implementation	20
11.	Experimental Evaluation and Results	24
12.	Comparison, Innovation and Scalability	27
13.	Conclusion and Future Work	28

1. INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

Financial markets generate large volumes of price data, technical indicators, news and other information that require both numerical analysis and contextual reasoning. LLMs can assist with financial analysis, but single-model systems may produce unsupported conclusions or overlook conflicting evidence.

AGENTTRADE addresses these limitations through a bounded multi-agent workflow involving evidence retrieval, Analyst reasoning, Bull-Bear debate, Critic evaluation and Correction. Deterministic software performs technical calculations and risk validation, while human approval is required before paper-trading execution.

The system therefore focuses on **evidence-based reasoning, traceability and risk-controlled AI-assisted stock analysis**.

1.2 NEED FOR AGENTIC AI

Retrieval-Augmented Generation (RAG) adds external evidence to model context [1]. Agentic reasoning introduces a choice of tools and queries instead of relying on one fixed context [2]. In this project, separate roles challenge an initial interpretation and a bounded correction pass revises it.

These features make the workflow easier to inspect, but they do not guarantee factual accuracy. Traces and source-support checks are still needed. Agentic AI is therefore used here as a design choice to be tested, not as proof of higher accuracy.

1.3 PROJECT OBJECTIVES

The primary objective of **AGENTTRADE** is to develop a bounded multi-agent framework for evidence-based stock research and risk-controlled paper trading, while separating probabilistic AI reasoning from deterministic risk validation and human-approved execution. The specific objectives are:

ID	Project Objective	Verification / Pass Condition
O1	Support evidence-based stock research and limited stock discovery	Market data, retrieved evidence and technical indicators are available for the tested research workflow
O2	Implement a transparent multi-agent reasoning workflow	Retrieval, Analyst, Bull, Bear, Critic and Correction stages complete successfully
O3	Compare different retrieval strategies	No-RAG, Fixed-RAG and Agentic-RAG are evaluated using the frozen 25-question benchmark
O4	Compare local language models	Qwen2.5 3B and Llama3.2 3B are evaluated under comparable benchmark conditions
O5	Enforce deterministic trading risk constraints	Quantity, notional, short-selling and paper-account capacity limits are validated independently of the language model
O6	Enforce human-controlled paper execution	Execution without explicit human approval is blocked, while an approved proposal reaches the paper-order path
O7	Verify the complete system workflow	Technical indicators, risk controls, human approval, smoke tests and the AAPL end-to-end paper-trading workflow are successfully verified

2. PROBLEM STATEMENT AND BUSINESS CASE

2.1 PROBLEM DEFINITION

Traditional stock-analysis workflows require users to collect market information, interpret technical indicators, evaluate supporting evidence and assess potential risks before making a trading decision. Large language models can assist with these tasks, but their outputs may contain unsupported reasoning, inconsistent conclusions or incorrect numerical interpretations. A single-model workflow also provides limited opportunity for structured disagreement and independent evaluation of an initial analysis.

AGENTTRADE addresses this problem by combining evidence retrieval with a multi-agent reasoning workflow consisting of an Analyst, Bull, Bear, Critic and Correction stage. The system separates probabilistic language-model reasoning from deterministic financial calculations and risk validation. A trade proposal generated by the agents is therefore treated as an advisory output rather than an executable instruction.

Before paper-order execution, the proposal is independently checked against configured quantity, notional, short-selling and paper-account constraints. Explicit human approval is then required before the proposal can reach the paper-trading path. This architecture is intended to improve the inspectability, controllability and safety of AI-assisted stock analysis without claiming predictive accuracy or profitability.

2.2 KEY CHALLENGES

Challenge	Problem	AGENTTRADE Approach
Unsupported AI reasoning	LLMs may generate conclusions without sufficient evidence	Retrieval stage provides supporting evidence
One-sided analysis	A single model may reinforce its initial conclusion	Bull-Bear debate introduces opposing arguments
Weak reasoning may remain uncorrected	Initial analysis may contain logical weaknesses	Critic identifies weaknesses and Correction revises the analysis
Numerical reliability	LLMs are not deterministic financial calculators	Technical indicators are calculated independently using Python
Unsafe trade proposals	Model-generated proposals may violate configured limits	Deterministic risk-validation layer checks the proposal
Uncontrolled execution	AI output should not directly trigger a trade	Explicit human approval is required
Benchmark ambiguity	Different retrieval strategies may produce different results	Frozen benchmark compares No-RAG, Fixed-RAG and Agentic-RAG

2.3 DESIGN PRINCIPLE

The central design principle of AGENTTRADE is “**AI reasons, deterministic code validates, and a human approves.**” The language-model agents are responsible for evidence interpretation, debate and recommendation generation. Deterministic Python components independently calculate technical indicators and enforce configured risk constraints. Paper-order execution remains behind an explicit human-approval boundary.

2.4 LIMITATIONS OF EXISTING APPROACHES

The following table compares a single conversational model with the proposed system. The AGENTTRADE column describes design intent and does not claim improved investment accuracy.

Aspect	Single conversational model	AGENTTRADE
Evidence	May answer confidently without showing the evidence behind the answer	Dated source cards and computed indicators retrieved from Qdrant
Challenge	Internal challenge is hard to inspect	Separate bull, bear, critic and one bounded correction role, visible in execution logs
Risk limits	Not enforced by the model	Deterministic Python checks: up to 5 shares, USD 1,000 notional, no short selling, 30-minute expiry
Execution	Analysis can blur into action	Research has no order path; only explicit human approval reaches the paper-order path
Error handling	Failures may be hidden behind fluent text	Errors and unconfigured states are shown directly

Table 2.1: Comparison of stock-research approaches

2.5 BUSINESS AND REAL-WORLD CONTEXT

AGENTTRADE can be used as a decision-support system for stock research and risk-controlled paper trading. It helps users analyze market information, compare bullish and bearish perspectives, and review risks before making a trading decision.

The system can support retail investors, financial researchers, and trading platforms by providing structured and evidence-based analysis. For real-world deployment, it would require stronger security, licensed financial data, operational controls, and financial/legal compliance. The current system is intended as a research prototype and is not an investment adviser.

The following table maps each problem to the corresponding capability of AGENTTRADE.

Table 2.2: Problems and the proposed solution

Problem	AGENTTRADE solution
Weakly supported recommendation	Dated source cards with fixed or agent-selected retrieval
Unfamiliar financial language	Readable summary separated from technical detail
Accidental jump from analysis to execution	Separate proposal and approval endpoints; research has no order path
Research, proposal and receipt confused	Distinct states for research, pending proposal, broker receipt and broker-owned portfolio
Data outage disguised as a missing stock	Errors and unconfigured states displayed directly

3. USER AND STAKEHOLDER ANALYSIS

3.1 TARGET USERS

The primary users of AGENTTRADE are retail investors, financial researchers and paper-trading users who require evidence-based stock analysis and controlled decision support. Academic evaluators and project developers are secondary stakeholders who assess the transparency, reliability and implementation of the system. A paper-account user can inspect research and choose whether to approve a simulated order.

3.2 STAKEHOLDER IDENTIFICATION

Stakeholder	Requirement / Need	How AGENTTRADE Supports It
Retail Investor	Understand a stock using multiple sources of evidence	Retrieval, Analyst and Bull-Bear analysis
Financial Researcher	Inspect reasoning and supporting evidence	Retrieved evidence, agent outputs and Critic evaluation
Paper-Trading User	Test proposed trades without real-money exposure	Alpaca paper-trading endpoint
System Administrator / Developer	Maintain and monitor the workflow	Modular agents, orchestration and monitoring components
Academic Evaluator	Verify system architecture and experimental claims	Benchmark results, verification tests and execution evidence

Table 3.1: Stakeholders and their key needs

3.3 FUNCTIONAL REQUIREMENTS

ID	Functional Requirement	Implementation / Component
FR1	Retrieve relevant stock-related evidence	Retrieval Agent / RAG pipeline
FR2	Obtain market information and calculate technical indicators	Market-data layer and Python calculation module
FR3	Generate an initial stock analysis	Analyst Agent
FR4	Generate opposing Bull and Bear arguments	Bull Agent and Bear Agent
FR5	Evaluate the quality and consistency of the debate	Critic/Judge Agent

FR6	Correct identified weaknesses in the analysis	Correction Agent
FR7	Generate a structured trade proposal	Proposal generation module
FR8	Validate trade proposals against deterministic constraints	Risk-validation engine
FR9	Prevent unauthorized paper execution	Human-approval gate
FR10	Submit approved orders to the paper-trading environment	Alpaca paper-trading endpoint
FR11	Compare retrieval and model configurations	Benchmark/evaluation module
FR12	Record and inspect system verification results	Evaluation and verification workflow

3.4 NON-FUNCTIONAL REQUIREMENTS

Requirement	Description
Safety	The system must remain within the configured paper-trading boundary and must not use a live-trading endpoint.
Reliability	Deterministic risk rules must be evaluated independently of language-model decisions.
Traceability	Research outputs should be associated with the evidence and reasoning stages used to produce them.
Reproducibility	Benchmark comparisons use a frozen question set and fixed evaluation configuration.
Modularity	Retrieval, reasoning, criticism, correction and risk validation should remain separate workflow stages.
Human Oversight	Paper-order execution must require explicit human approval.
Maintainability	Agent and deterministic validation components should be independently testable and replaceable.
Bounded Execution	Configured quantity, notional, short-selling and paper-account capacity limits must be enforced.

3.5 USER REQUIREMENTS

Functional requirements:

- Accept a natural question in a single research box; detect common company names and explicit tickers.
- Ask for a ticker when the input is unresolved instead of silently choosing a default, and ask for clarification when multiple distinct stocks are named.
- Start a named-stock request without an extra confirmation step.
- For open-ended discovery, present one candidate and up to two alternatives, with a warning that only a small screen was considered.
- Separate a readable summary from technical detail, and keep portfolio information, pending proposals and approval controls distinguishable.

Non-functional requirements:

- Transparency — errors and unconfigured states must be shown directly. A data outage must not be disguised as a missing-stock question, and absent paper credentials must not create an apparently executable proposal.
- Security hygiene — local services should stay bound to the host, and secrets should not appear in reports or source control.
- Traceability — execution traces should be retained for evaluation.

4. PROPOSED AGENTIC AI SOLUTION

4.1 OVERALL SOLUTION

AGENTTRADE is a bounded multi-agent framework designed for evidence-based stock research and risk-controlled paper trading. The system separates its workflow into distinct stages, allowing language models to perform research and reasoning while deterministic software handles numerical validation and risk checks. The workflow begins with market-data and evidence retrieval, followed by Analyst interpretation and an explicit Bull-Bear debate. A Critic/Judge evaluates the competing arguments, and a Correction Agent address identified weaknesses. The resulting trade proposal is then processed by a deterministic risk-validation layer, and only proposals satisfying the configured constraints can proceed to the human-approval stage. After explicit approval, the order is sent only to the configured Alpaca paper-trading endpoint. This separation ensures that an AI-generated recommendation remains an advisory proposal rather than an executable trading instruction.

4.2 KEY FEATURES

- **Evidence-based retrieval:** Relevant market and supporting information is retrieved before the reasoning stages.
- **Multi-agent analysis:** Separate agents perform analysis, Bull-Bear debate, criticism and correction.
- **Structured disagreement:** Bull and Bear agents provide opposing perspectives before a final evaluation.
- **Critic-based validation:** A dedicated Critic/Judge identifies weaknesses or inconsistencies in the proposed reasoning.
- **Bounded correction:** The Correction Agent revises the analysis using the identified issues rather than generating an unrestricted new decision.
- **Deterministic technical analysis:** Technical indicators are calculated independently using Python-based numerical logic.
- **Deterministic risk validation:** Trade proposals are checked against quantity, notional, short-selling and paper-account constraints.
- **Human approval:** Explicit user approval is required before a proposal reaches the paper-order path.
- **Paper-only execution:** The configured execution endpoint is the Alpaca paper-trading environment, with no live-trading endpoint enabled.
- **Retrieval benchmarking:** No-RAG, Fixed-RAG and Agentic-RAG configurations are compared using a frozen benchmark.

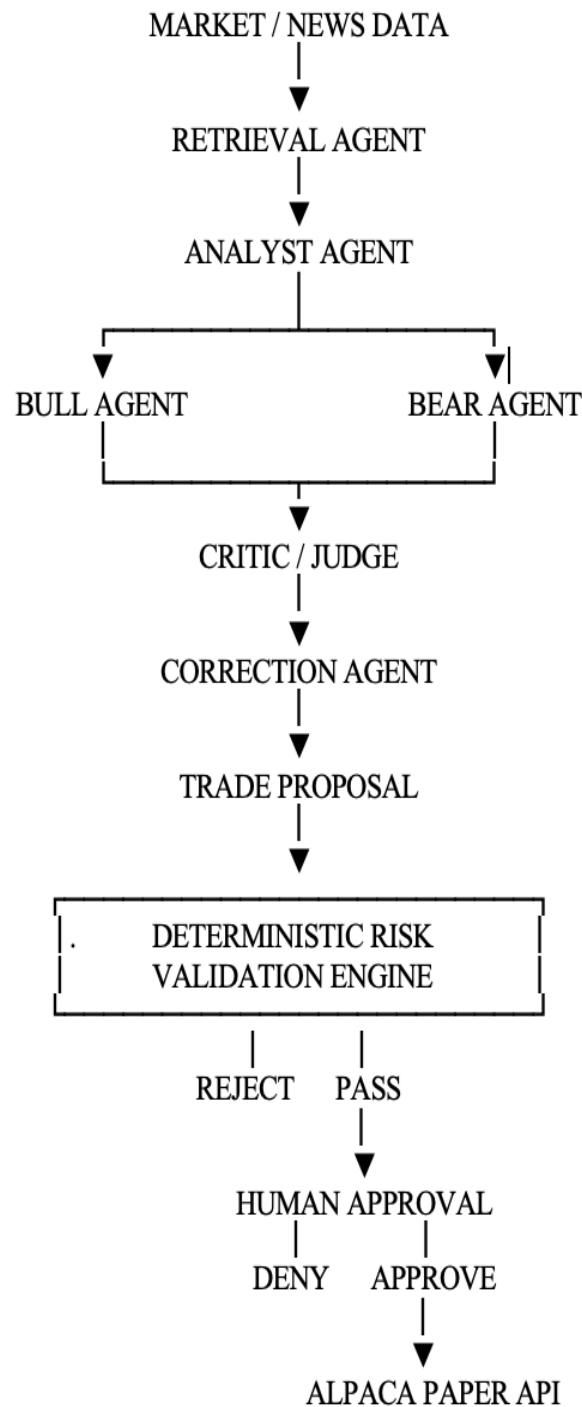
4.3 AGENT-BASED APPROACH

AGENTTRADE decomposes stock analysis into specialised reasoning roles rather than relying on a single language-model response. The Retrieval Agent supplies relevant evidence, while the Analyst Agent interprets the available information. The Bull and Bear Agents then construct opposing arguments, allowing potential weaknesses and alternative interpretations to become explicit. The Critic/Judge Agent evaluates the debate and identifies weaknesses, after which the Correction Agent produces a bounded revision. The resulting proposal is passed outside the language-model reasoning chain to deterministic risk validation and, only after explicit human approval, to the paper-trading path.

Agent / Component	Primary Responsibility	Output	Authority
Retrieval Agent	Retrieve relevant evidence	Evidence set	Advisory
Analyst Agent	Interpret evidence and indicators	Initial analysis	Advisory
Bull Agent	Construct positive investment argument	Bull argument	Advisory
Bear Agent	Challenge the positive argument	Bear argument	Advisory
Critic/Judge Agent	Evaluate competing arguments	Critique / evaluation	Advisory
Correction Agent	Address identified weaknesses	Corrected analysis	Advisory
Risk Engine	Validate proposal constraints	Accept / Reject	Deterministic authority
Human Approval	Approve or reject execution	Approval / rejection	Human authority
Alpaca Paper Endpoint	Process approved paper order	Paper-order result	Paper execution

4.4 AI-RISK-EXECUTION AUTHORITY BOUNDARY

A key architectural principle of AGENTTRADE is the separation of reasoning authority from execution authority. The language-model agents can generate, compare and revise analytical proposals, but they cannot directly authorize an order. Deterministic Python logic independently validates the proposal against the configured risk constraints. A valid proposal must then pass an explicit human-approval gate before reaching the paper-trading endpoint. Consequently, the system follows the principle: **AI reasons** → **deterministic code validates** → **human approves** → **paper system executes**.



5. DATASET AND KNOWLEDGE SOURCES

AGENTTRADE combines a frozen synthetic benchmark corpus with live public market data, external information sources and a paper-trading environment. The synthetic benchmark is used for controlled comparison of language models and retrieval strategies, while live data and paper-broker integration are evaluated separately during system-level verification. This separation ensures that benchmark results are not interpreted as evidence of real-world financial performance.

Table 5.1: Knowledge sources and limits

Source	Contents / Coverage	Used For
Synthetic AAPL/MSFT Benchmark Corpus	Frozen stock-related facts, evidence cards and policies used for the controlled benchmark	RAG and model comparison
Market Data	Stock price and market information used by the research workflow	Stock analysis and technical indicators
NewsAPI	External news information relevant to stock research	News/evidence retrieval
Qdrant Knowledge Base	Indexed evidence/source cards used by retrieval workflows	Fixed-RAG and Agentic-RAG
Alpaca Paper Trading	Paper account and paper-order endpoint	Execution-path verification
Technical Indicator Data	Price inputs used for SMA, RSI and MACD calculations	Deterministic indicator verification

The synthetic benchmark corpus is intentionally separated from live market information. It provides a frozen and repeatable evaluation environment for comparing No-RAG, Fixed-RAG and Agentic-RAG configurations across Qwen2.5 3B and Llama3.2 3B. Live market data, NewsAPI information and Alpaca paper trading are instead used to verify the operational workflow of the implemented system.

Preparation and indicators. Python computes RSI14, SMA20/50, MACD and its signal line from price history. Cards retain source and date metadata. Nomic embeddings represent the text in Qdrant, and ticker tags allow company-scoped retrieval. A headline is not a full article and does not verify the underlying claim.

Data quality and reproducibility. Missing news produces a flagged price/statistics fallback. Synthetic policy card 3 contains an obsolete two-ticker restriction and is annotated as historical; current API rules define the live scope. The frozen benchmark should run on a clean Qdrant collection before live research, because ingest does not remove every previously added live card and mixed data may change the comparison. A formal experiment should preserve model versions, prompts, embeddings, the corpus and retrieval traces.

5.2 POINT-IN-TIME HANDLING

The benchmark evaluation uses a frozen synthetic corpus so that the same evidence and questions can be presented consistently across the evaluated models and retrieval configurations. The benchmark corpus is kept separate from live market data used during system-level verification. This separation prevents live verification data from being incorporated into the controlled benchmark and allows the reported retrieval results to remain reproducible for the defined evaluation setup.

Table 5.2: Data Usage and Evaluation Boundary

Data Category	Purpose	Evaluation Role
Synthetic benchmark corpus	Controlled stock-research questions and evidence	Model/RAG benchmark
Live market data	Runtime stock analysis	System verification
NewsAPI data	Runtime news retrieval	System verification
Alpaca paper account	Paper-order processing	Execution verification

The benchmark results and live-system verification results are therefore reported as separate forms of evidence: the benchmark evaluates model and retrieval behaviour, while live-system verification evaluates the implemented workflow and execution controls.

6. TOOLS AND TECHNOLOGIES USED

AGENTTRADE integrates local AI models, multi-agent orchestration, retrieval infrastructure, financial data services, a lightweight backend, and a browser-based user interface. The technology stack is designed to separate probabilistic language-model reasoning from deterministic financial calculations, risk validation, data retrieval, and paper-trading execution.

Table 6.1: Technology stack

Category	Technology	Purpose
Agent Orchestration	n8n	Orchestrates the multi-agent workflow, including Retrieval, Analyst, Bull, Bear, Critic and Correction stages.
Local LLM Runtime	Ollama	Runs open-source language models locally for agent reasoning and generation.
Language Models	Qwen 2.5 (3B/7B)	Generates research analysis, debate arguments and decision-oriented responses.
Vector Database	Qdrant	Stores and retrieves embedded evidence and news information for RAG.
Programming Language	Python 3	Implements backend logic, data processing, technical indicators and deterministic risk validation.
Backend Framework	FastAPI	Provides backend APIs, routing and application services.
Web Server	Uvicorn	Runs the FastAPI application using the ASGI interface.
Database	SQLite3	Stores application data including users, trade proposals and paper-trading records.
Market Data	yfinance	Provides historical market data used for stock analysis and technical indicators.
News Data	NewsAPI	Retrieves recent news headlines and related evidence for the research workflow.
Paper Trading	Alpaca API	Provides paper-account information, portfolio tracking and paper-order execution.
Frontend	HTML5, CSS3, JavaScript	Implements the browser-based user interface and dashboard.
Visualization	Chart.js	Renders interactive portfolio and P&L visualizations.
Real-Time Communication	Server-Sent Events (SSE)	Streams agent workflow and execution status from the backend to the frontend.
Containerization	Docker	Provides isolated and reproducible execution environments for project services.
Container Orchestration	Docker Compose	Coordinates services such as n8n and Qdrant.
Testing	pytest	Supports automated testing of application components and risk-control logic.
Automation	Bash / start_all.sh	Automates the startup sequence of the application and supporting services.

The technologies were selected to provide a modular architecture in which language models perform probabilistic reasoning while deterministic software handles numerical calculations, risk validation and execution constraints. n8n manages the agent workflow, Ollama provides local model inference, and Qdrant supports evidence retrieval. FastAPI and SQLite3 provide the application backend and persistent storage, while yfinance, NewsAPI and Alpaca provide market, news and paper-trading functionality. The browser-based interface uses HTML, CSS, JavaScript and Chart.js, with Server-Sent Events providing real-time workflow updates.

Deployment responsibilities. Docker builds the API from pinned requirements. Compose publishes ports 8000 and 6333 on 127.0.0.1 only. Native n8n typically uses port 5678 and Ollama uses 11434. Mac and Windows require the same models, with host-specific native setup. n8n provides orchestration [3], Ollama serves local inference [4], and Qdrant provides vector retrieval [5].

Practical constraints. Local models require RAM, storage and inference time. Chart and Markdown assets load from CDNs, so the interface is not fully offline. Market data and broker calls also need the internet. The local services should not be exposed publicly; the current secret and identity design is not suitable for that use.

6.2 Technology Selection and Design Rationale

The technology stack of AGENTTRADE was selected according to the functional requirements of multi-agent reasoning, evidence retrieval, financial analysis, deterministic risk validation and controlled paper trading. Local LLM execution through Ollama reduces dependence on external model APIs and allows the reasoning components to operate within the project's local environment. n8n provides visual orchestration of the agent workflow, making the sequence of retrieval, analysis, Bull-Bear debate, Critic evaluation and Correction explicit and traceable.

Qdrant was selected as the vector database for storing and retrieving evidence used by the RAG pipeline. Python provides the deterministic computational layer for technical indicators and risk checks, while FastAPI exposes the backend services required by the application. SQLite3 provides lightweight persistent storage for application and trading records.

Financial data is obtained through yfinance and NewsAPI, while Alpaca is restricted to paper-trading operations for execution verification. The frontend uses standard web technologies with Chart.js for visualization, and Server-Sent Events provide real-time updates of the agent workflow. Docker and Docker Compose provide reproducible service deployment, while pytest supports automated verification of application and risk-control components.

This technology selection follows the central architectural principle of AGENTTRADE: **AI reasons, deterministic software validates, and a human approves execution.** The resulting modular structure allows individual components to be tested, replaced or extended without requiring a complete redesign of the system.

7. AGENT ARCHITECTURE

AGENTTRADE uses a structured multi-agent architecture in which each agent has a defined responsibility and authority boundary. The agents do not independently execute trades. Instead, they progressively transform retrieved evidence into an evaluated trade proposal, which is subsequently checked by deterministic risk controls and requires explicit human approval before paper execution.

7.1 AGENT COMPONENTS

Table 7.1: Agents, models and responsibilities

Agent	Primary Input	Primary Output	Authority / Responsibility
Retrieval Agent	Stock query, market/news sources, Qdrant knowledge base	Relevant evidence and source context	Retrieves evidence; does not make trading decisions
Analyst Agent	Retrieved evidence, market data, technical indicators	Structured stock analysis	Interprets evidence and produces the initial analysis
Bull Agent	Analyst output and supporting evidence	Bullish argument	Constructs the strongest evidence-based bullish case
Bear Agent	Analyst output and supporting evidence	Bearish argument	Constructs the strongest evidence-based bearish case
Critic / Judge Agent	Analyst output, Bull argument and Bear argument	Evaluation, weaknesses and preferred conclusion	Compares competing arguments and identifies unsupported or weak reasoning
Correction Agent	Critic findings, previous analysis and evidence	Corrected analysis / final research output	Addresses identified weaknesses and produces a refined result

The architecture intentionally separates research responsibilities across agents rather than allowing a single language-model call to generate the complete decision. The Retrieval Agent establishes the evidence context, while the Analyst interprets the retrieved information. The Bull and Bear Agents provide opposing perspectives to reduce one-sided reasoning. The Critic/Judge evaluates the competing arguments, and the Correction Agent incorporates the identified weaknesses into the refined output. The resulting research output is still treated as a proposal rather than an executable trading instruction. Deterministic risk validation and explicit human approval remain outside the language-model decision process.

SYSTEM ARCHITECTURE

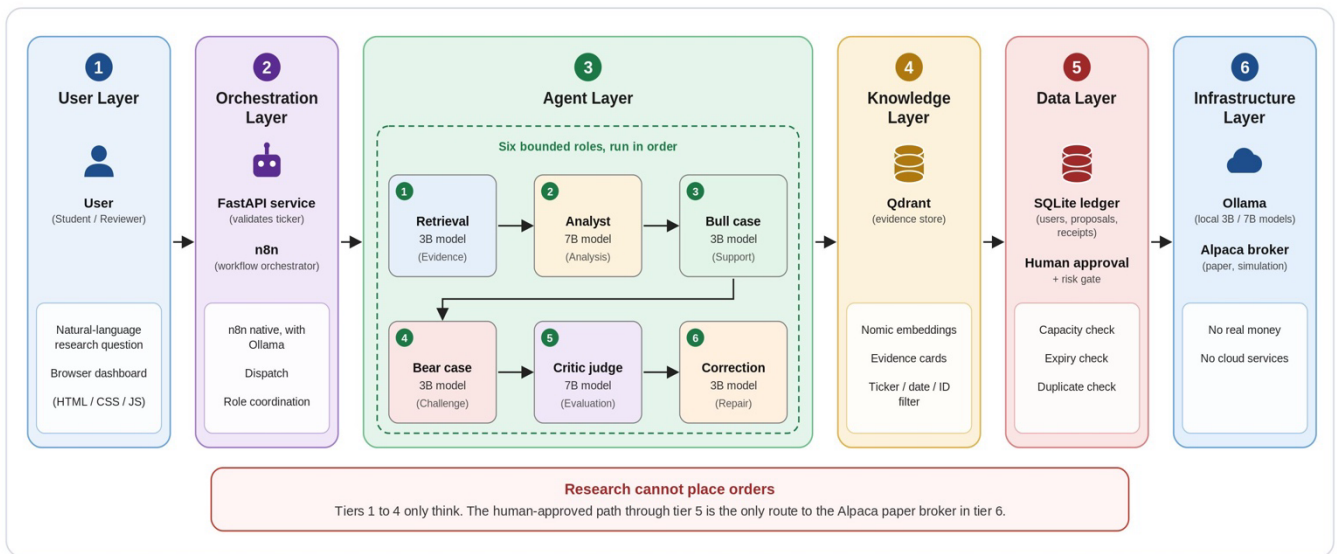


Figure 7.1: System components and the independent paper-execution boundary. Stores support the service and workflow; they are not autonomous trading actors.

Agent authority boundary: No research agent has direct authority to bypass deterministic risk validation or human approval and execute a paper trade.

7.2 Detailed Agent Reasoning Flow

The AGENTTRADE reasoning process is designed as a staged decision-support pipeline rather than a single model response. Each stage contributes a specific form of reasoning, while the overall workflow preserves the evidence and intermediate outputs required for inspection.

Table 7.2: Detailed Agent Reasoning Process

Step	Stage	Reasoning Activity	Result
1	Evidence Retrieval	Identifies relevant market information, news and stored evidence for the requested stock	Evidence context
2	Analyst Reasoning	Interprets retrieved evidence together with technical indicators and market information	Initial analysis
3	Bull Analysis	Develops the strongest evidence-supported case for a positive outlook	Bull argument
4	Bear Analysis	Develops the strongest evidence-supported case for a negative outlook	Bear argument
5	Critic / Judge	Compares the two arguments, identifies weaknesses, unsupported claims and conflicting evidence	Critical evaluation
6	Correction	Revises the analysis using the Critic's findings and available evidence	Refined research output
7	Trade Proposal	Converts the refined analysis into a bounded candidate proposal when appropriate	Proposed action
8	Deterministic Validation	Independently checks quantity, notional value, short-selling restrictions and paper-account capacity	Approved/rejected proposal
9	Human Approval	Requires explicit user approval before execution	Approval decision
10	Paper Execution	Sends an approved proposal to the configured Alpaca paper-trading endpoint	Paper-order result

A key design characteristic of AGENTTRADE is the separation between reasoning and authority. The language-model agents can interpret evidence, construct arguments and revise conclusions, but their outputs do not directly authorize a trade. Numerical and policy constraints are evaluated by deterministic software after the research process. Only a proposal that passes these checks can reach the human-approval stage, and only an explicitly approved proposal can proceed to the configured paper-trading endpoint.

7.3 REASONING FLOW

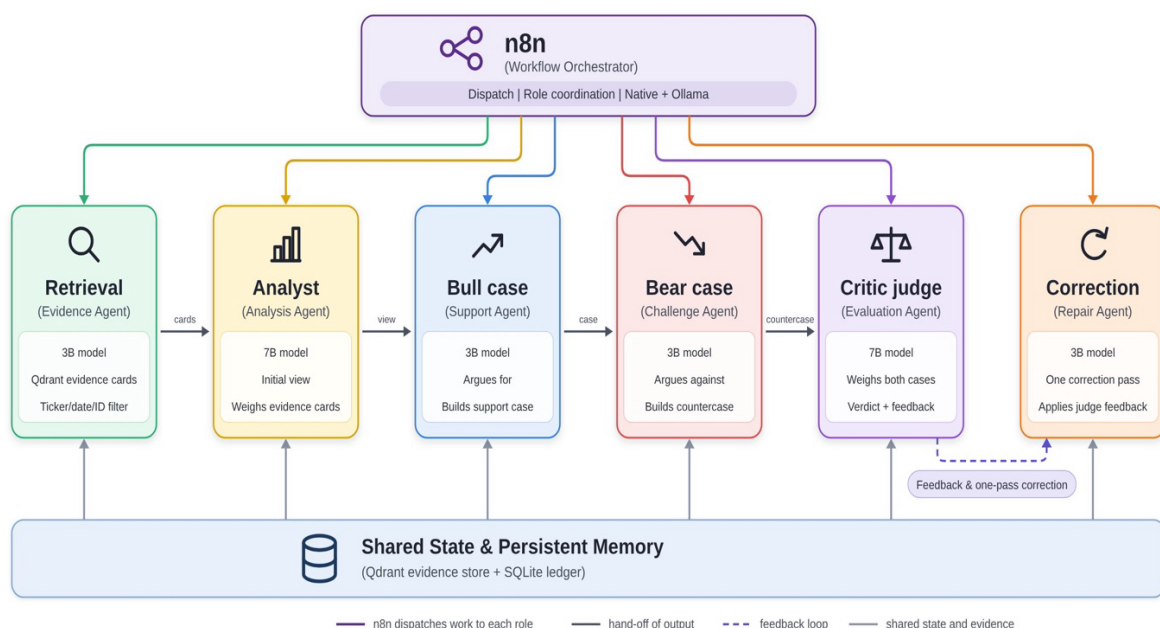
Separate roles make challenge and critique visible in the execution logs. A single agent would be simpler and faster, but its internal challenge would be harder to inspect. More roles also add latency and can repeat the same errors. Debate should therefore be treated as a design choice to test, not as proof of higher accuracy.

Correction is limited to one pass. Resolved model tiers are returned for audit, while legacy single-model requests support the benchmark. A 3B correction model can alter a 7B answer, so the interactive split needs its own evaluation; the published single-model results do not answer that question.

Current answer-format limitation. The parser accepts Markdown headings but can also accept missing sections with the fallback summary “Analysis Complete”. It overwrites a mismatched separate summary. This differs from earlier strict-validation descriptions and does not establish that every answer is readable or consistent. Clear summary validation and honest error handling remain necessary.

7.4 AGENT INTERACTIONS

The API resolves the question and prepares the evidence context; n8n then sequences retrieval, analysis, bull-bear debate, critique and correction. The research agents never call the broker. Proposal creation and approval are separate user actions, so the research workflow cannot directly place an order, even when its conclusion is wrong.



8 AGENT WORKFLOW AND ORCHESTRATION

8.1 WORKFLOW DESIGN

Named-stock research and small-candidate discovery share a bounded research pipeline, shown in Figure 8.1. A named-stock request bypasses the candidate screen; an open-ended request first passes through the discovery screen. Prepared market and evidence context then flows through the retrieval agent, analyst, bull and bear debate, judge and one correction pass before the research is presented.

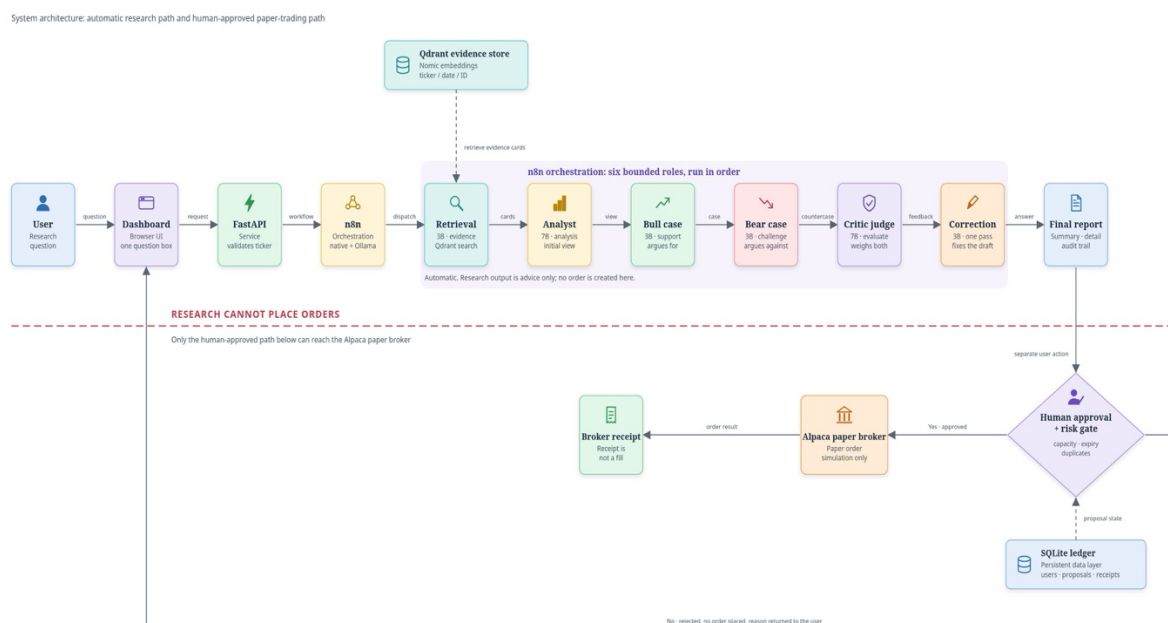


Figure 8.1: Workflow design — named-stock research and small-candidate discovery.
Discovery uses fixed retrieval under the current browser selection mapping.

8.2 TASK DECOMPOSITION

The API resolves the question: a company or ticker leads to research, while an open-ended question leads to discovery. For discovery, a few most-active and top-gainer candidates are screened, each is researched, and the verdict and technical score rank them into one pick and up to two backups. Evidence preparation validates the data and routes the models for no retrieval, fixed retrieval or agent-selected retrieval.

8.3 AGENT COORDINATION

n8n sequences retrieval, analysis, bull-bear debate, critique and correction. Timed progress pills are a user-interface aid, not evidence that a workflow node has completed. Actual traces establish tool and model execution.

8.4 ORCHESTRATION IMPLEMENTATION

n8n is used as the orchestration layer for coordinating the AGENTTRADE workflow. It manages the sequence of retrieval, analysis, Bull-Bear debate, Critic evaluation, Correction, proposal generation and subsequent validation stages. The backend communicates with the workflow through defined API/webhook interfaces, while intermediate outputs are passed between stages as structured data. This design keeps the agent workflow modular and allows individual stages to be tested independently.

9 AGENTIC RAG IMPLEMENTATION

9.1 RETRIEVAL ARCHITECTURE

Source cards are embedded with Nomic and stored in Qdrant with IDs, dates and ticker metadata; ticker tags allow company-scoped retrieval. Three retrieval conditions are compared, as summarised in Table 9.1.

Table 9.1: Controlled retrieval comparison

Condition	Retrieval and generation
No RAG / none	Bypass retrieval; no external evidence is supplied.
Basic RAG / fixed	Python retrieves fixed top-3 context as fixed_evidence.
Agentic RAG	The retrieval agent formulates a query and uses Qdrant; query reformulation is possible.

Data flow

How evidence, proposals and portfolio state move through the system. Research output is advice only; an approved proposal is the only route to the paper broker.

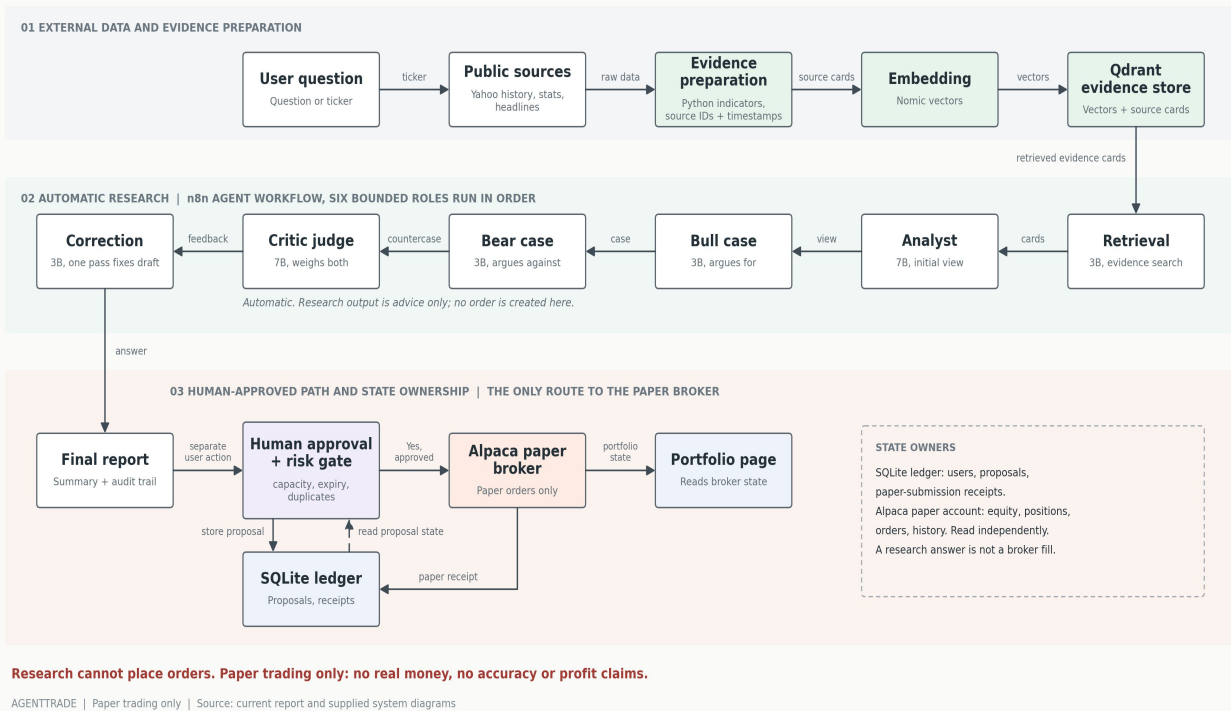


Figure 9.1: Evidence preparation, local proposal state and broker-owned state serve different purposes.

9.2 KNOWLEDGE RETRIEVAL

The retrieval component uses a **synthetic benchmark case used only for controlled evaluation**. Evidence is stored as structured source cards in Qdrant and retrieved according to the selected retrieval mode. The retrieved context is then provided to the downstream Analyst and debate agents for evidence-based reasoning.

9.3 GENERATION PROCESS

Generation follows the bounded role sequence. The analyst interprets the retrieved evidence and indicators and prepares an initial view; the bull and bear roles develop independent opposing cases; the critic judge assesses support, conflicting cases and the decision; and one correction pass revises the answer using the critic feedback. The result is returned through the API as a summary, detail and audit fields. Research output is never a broker fill.

9.4 RETRIEVAL-QUALITY EVALUATION

Term accuracy, citation presence and a rationale proxy do not directly measure source relevance or expert financial accuracy. Human annotation and traces are needed before deciding that a retrieval policy is better. In this project the three conditions are compared on the same 25 frozen questions, as described in Section 1.

10 SYSTEM IMPLEMENTATION

10.1 FUNCTIONAL PROTOTYPE SCREENSHOTS

The AGENTTRADE prototype provides a research dashboard for stock queries, retrieval selection, agent reasoning, trade proposals and portfolio information. The implemented workflow was verified through an end-to-end AAPL test covering evidence retrieval, Analyst reasoning, Bull-Bear debate, Critic evaluation, Correction, proposal generation, risk validation, human approval and paper-order submission.

The system uses the configured Alpaca paper-trading endpoint only. The verification demonstrates workflow execution and paper-order transmission; it does not establish real-money trading, profitability or predictive accuracy.

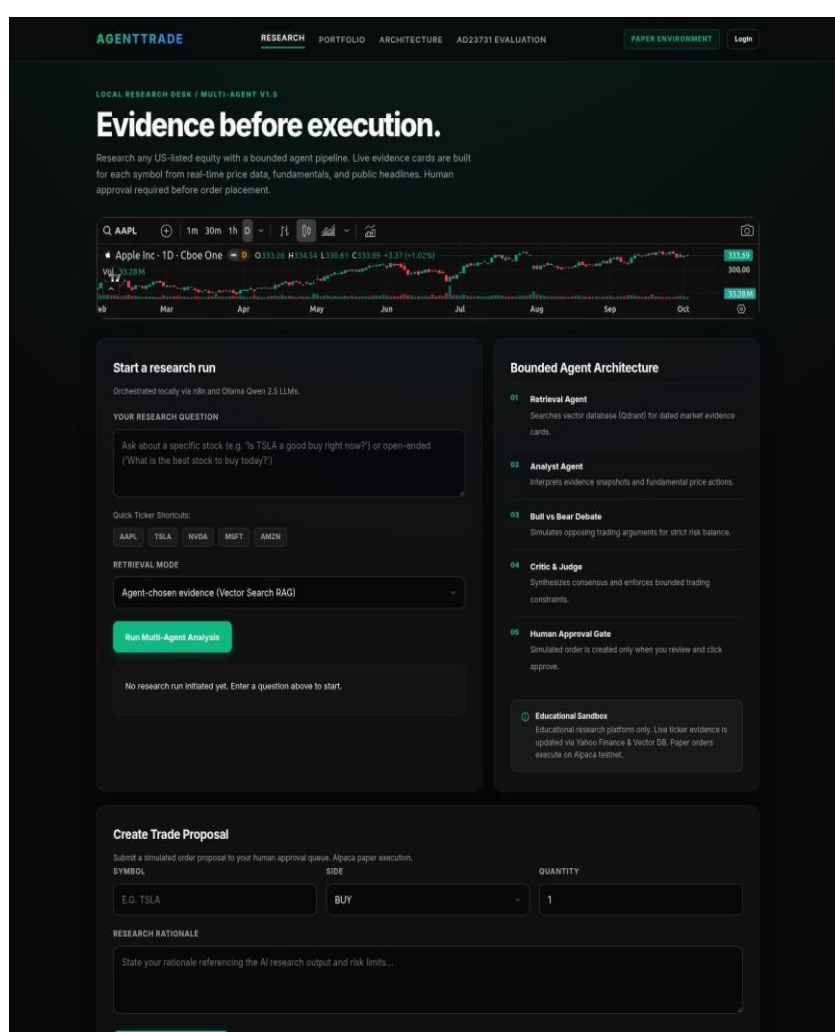


Figure 10.1: Upper portion of the actual dashboard initial state, locally rendered on 4 October 2026. This is an interface screenshot, not a completed model run.

Portfolio and local credentials. The /api/portfolio endpoint reads broker equity, cash, buying power, positions, recent orders and history. A submission receipt is not fill-confirmed performance, and new accounts may have no chart history. The screenshot below therefore establishes the unconfigured interface state, not a return or profit.

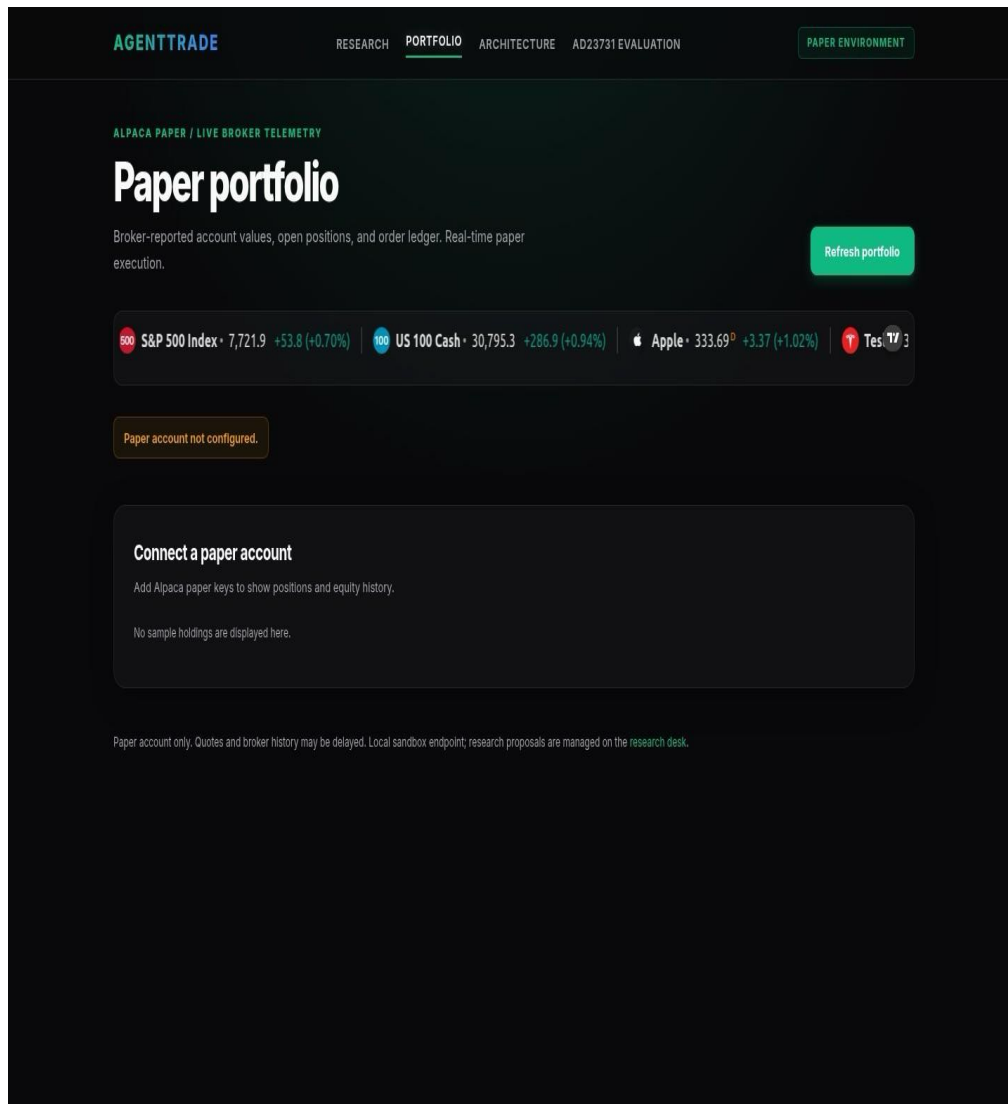


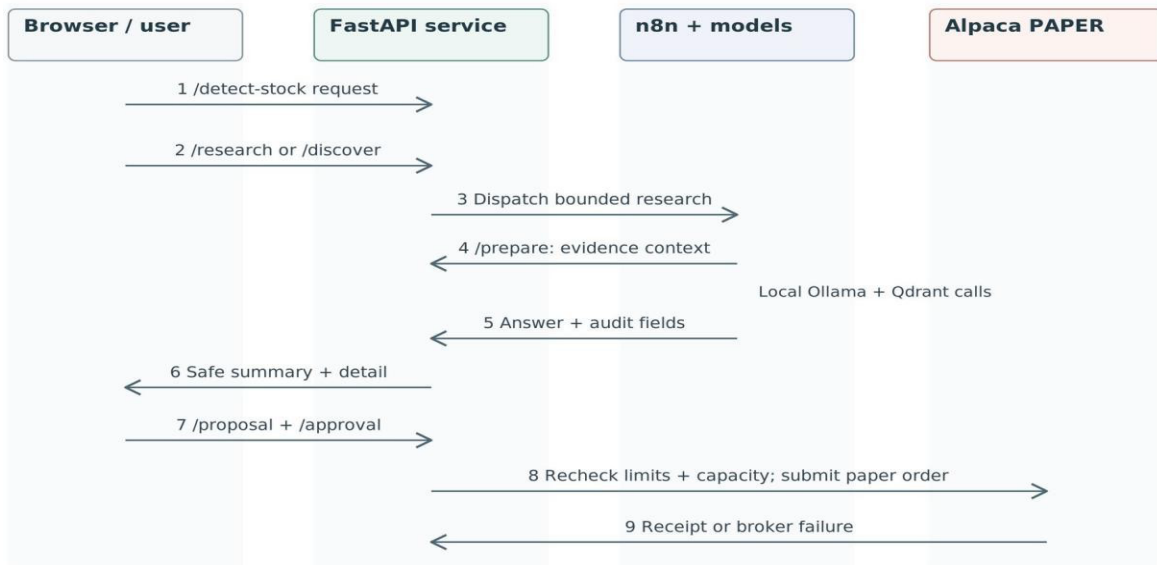
Figure 10.2: Actual paper-portfolio unconfigured state. No sample holdings or investment returns are asserted.

The dashboard includes registration, login and paper-key setup. Passwords and per-user keys are stored in plaintext SQLite. Login does not establish complete endpoint session authorization, and filtering a ledger by username is not an access-control boundary. Keep the host private and localhost-only. Reports, Git and screenshots must not contain actual credentials.

End-to-end request sequence. The research request returns a summary, detail and audit fields through the API. A separate user choice creates a proposal and reaches approval. Portfolio reads obtain broker state independently of research output.

End-to-end request sequence

Research and paper approval are separate request paths. Portfolio reads are independent.



Portfolio reads broker state independently. This is a source sequence, not a verified live order.

AGENTTRADE | Paper trading only | Source: current report and supplied system diagrams

Figure 10.3: Browser, API, workflow and broker request sequence. This describes source behavior, not a verified live broker submission.

Human review and independent risk controls. The user should inspect the ticker, side, quantity, rationale and account capacity before choosing approval. Model confidence is not permission. Proposal creation requires configured paper keys and capacity checks; the earlier no-key local proposal behavior is not supported. Limits include up to five shares, USD 1,000 notional, no short selling, 30-minute expiry and duplicate checks. Approval repeats the relevant checks, including current buying power and holdings, and surfaces broker failures instead of reporting a false success.

Human approval and paper-order gate

30-minute expiry | Maximum 5 shares and USD 1,000 notional | No short selling

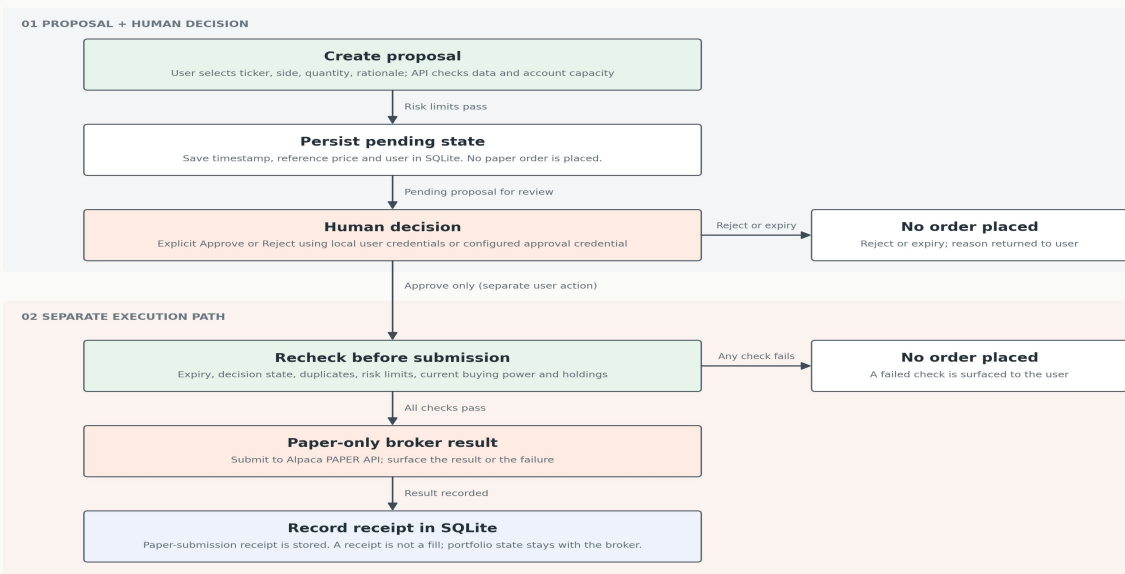


Figure 10.4: Independent human-approval and paper-order path, including expiry, duplicate and account-capacity checks.

Alpaca paper trading is used to simulate orders [6]. No evidence in this report establishes a real broker submission, a filled order or profitable trading. The paper-only boundary must remain in place during local validation.

Rendering patch. The implementation escapes model and news HTML, permits inert formatting and removes active attributes. A harmless local image-event regression test no longer executed after that patch. This was a narrow rendering fix, not a full security audit.

10.2 SOURCE-CODE REPOSITORY

The source code, Docker Compose configuration, n8n workflow JSON files (research.json and research_clean.json) and evaluation scripts are kept together with the project. Repository link: https://github.com/Doneeswaran/Foundation_of_Agentic_AI/tree/main . Credentials must not be committed to the repository.

11 EXPERIMENTAL EVALUATION AND RESULTS

11.1 EVALUATION METRICS

Table 11.1: Evaluation metrics

Metric	Meaning and Caution
Label-term accuracy	Expected-term matching; a wrong answer may still contain the correct term.
Reasoning proxy	Citation/rationale heuristic, not expert grading.
Citation rate	Detected citation syntax, not verified source support.
Exact consistency	Normalized answer equality; paraphrases may score zero.
Success / errors	Request completion counts; HTTP success is not answer correctness.
Latency	Timing is recorded by the evaluator, but no raw verified timing table was supplied.
Technical-indicator correctness	Verification of deterministic SMA, RSI and MACD calculations.
Risk-rule enforcement	Verification of quantity, notional, short-selling and paper-capacity constraints.
Human-approval enforcement	Confirms unauthorized execution is blocked and approved execution proceeds.
End-to-end workflow completion	Confirms the complete retrieval-to-paper-trading workflow.

11.2 TESTING METHODOLOGY

The experiment uses 25 fixed questions, two single-model configurations, three retrieval modes and three repeats: $25 \times 2 \times 3 \times 3 = 450$ workflow requests. Each request includes several model calls. Qwen2.5 3B is compared with Llama3.2 3B; this is not an evaluation of the interactive 7B/3B split.

The same frozen cards, model versions, prompts and embeddings should be used, starting from a clean collection, with answers preserved and retrieval traces inspected. Source support should be annotated manually. A one-repeat pipeline smoke run cannot establish response consistency.

11.3 PERFORMANCE RESULTS

The project report records a run start of 3 October 2026 at 16:24:59 UTC and six rows totaling 450 successful requests. The raw CSV, summary JSON and traces were not supplied. The values in Table 11.2 are reported, not independently rerun or verified, and no model benchmark was run for this restyled report.

Table 11.2: Reported results, with scores on a 0–1 scale

Model	Mode	OK / total	Accuracy	Reason.	Citation	Consist.
Qwen 3B	none	75 / 75	0.307	0.000	0.013	0.000
Qwen 3B	fixed	75 / 75	0.707	0.000	0.000	0.000
Qwen 3B	agentic	75 / 75	0.640	0.013	0.053	0.000

Llama 3B	none	75 / 75	0.293	0.000	0.000	0.000
Llama 3B	fixed	75 / 75	0.893	0.000	0.000	0.000
Llama 3B	agentic	75 / 75	0.747	0.027	0.027	0.000

Models: qwen2.5:3b and llama3.2:3b. Reported errors are zero per row. No latency or profit values were supplied.

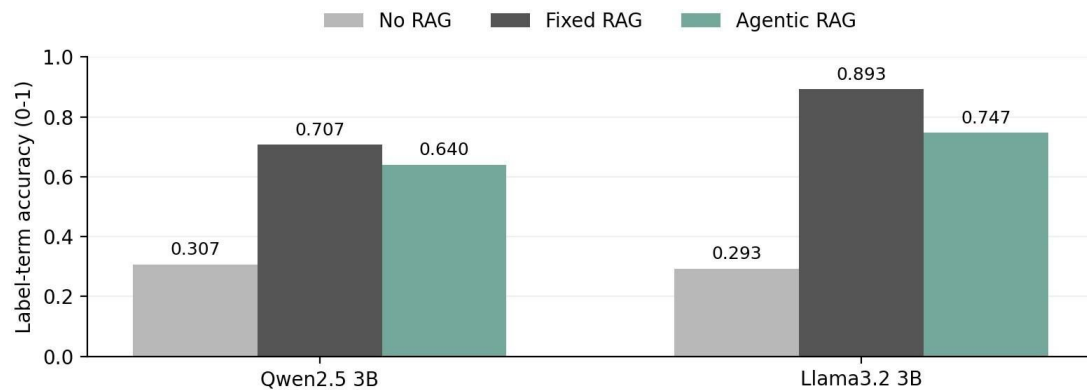


Figure 11.1: Label-term accuracy calculated from Table 11.2. The graph is not a backtest or a measure of financial return.

Fixed RAG has higher reported term accuracy than agentic RAG for both models: 0.707 versus 0.640 for Qwen and 0.893 versus 0.747 for Llama. No-RAG is lower. The table does not support a general claim that agentic retrieval outperforms fixed retrieval.

Citation rates and reasoning proxies are low, and exact consistency is 0.000 in every row. Formatting, metric design or actual instability may contribute; raw answers and traces are needed to distinguish them. The small synthetic task set is not investment evidence.

Table 11.3: Executed System Verification

Check	Verified Outcome
Technical indicators	SMA20, SMA50, RSI14 and MACD matched the expected deterministic baseline values.
Risk-control engine	Quantity, notional, short-selling and paper-capacity constraints were verified.
Human approval — bypass path	Unauthorized execution was blocked.
Human approval — approved path	A valid approved proposal reached the paper-order path.
AAPL end-to-end workflow	Retrieval, Analyst, Bull, Bear, Critic, Correction, proposal and paper-trading stages completed.
Smoke testing	Portfolio, NewsAPI and evaluator smoke checks passed.

Paper broker	Alpaca paper-trading endpoint was used; no live-trading endpoint was enabled.
--------------	---

All executed verification tests passed for the tested scenarios. The verification covers deterministic technical-indicator calculations, configured risk boundaries, human-approval enforcement, the end-to-end AAPL workflow and smoke-test functionality. These results demonstrate reliability of the tested implementation paths; they do not establish predictive investment accuracy, profitability or production-grade security.

11.4 SUCCESS-METRIC ACHIEVEMENT

Table 11.4: Success criteria

Success Criterion	Expected Outcome	Achieved
Retrieval benefit	Retrieval raises label-term accuracy over No-RAG	Yes — reported: 0.707 vs 0.307 Qwen; 0.893 vs 0.293 Llama
Agentic advantage	Agentic RAG scores above Fixed RAG	No — 0.640 vs 0.707 Qwen; 0.747 vs 0.893 Llama
Execution safety boundary	Research cannot place orders; approval is separate	Yes — risk validation and human approval verified
Citation reliability	Claims linked to verified sources	Not established — citation rate measures syntax only
System verification	Tested implementation paths complete successfully	Yes — executed verification scenarios passed
End-to-end runtime	Retrieval → reasoning → approval → paper trading verified	Yes — AAPL workflow verified through paper-order path
Efficiency	Latency and cost within acceptable limits	Not measured — no verified latency values

12 COMPARISON, INNOVATION AND SCALABILITY

12.1 COMPARISON WITH BASELINE

No-RAG isolates model-only answers. Fixed RAG tests deterministic top-3 context, while agentic RAG tests query choice and tool use. Quality and cost should be compared empirically; the current table favors fixed retrieval on strict label-term accuracy. The quantitative comparison is presented in Section 11.

12.2 ADVANTAGES OF THE PROPOSED SYSTEM

- Local inference reduces remote-model disclosure, although data and broker calls remain external.
- Separating research from approval reduces accidental execution, but identity and secret handling still need work.
- Bounded correction is inspectable, while multiple model calls increase latency.
- A readable interface helps review; timed progress cannot replace authoritative execution traces.

12.3 INNOVATION

Inspectable multi-agent reasoning + deterministic risk validation + mandatory human approval + paper-only execution.

- Inspectable role separation — retrieval, analyst, bull, bear, critic and one bounded correction.
- Live ticker evidence alongside a frozen synthetic benchmark corpus.
- Deterministic risk checks independent of the language models.
- Human approval as the only route to a paper order.
- Discovery that reduces a broad question to a small candidate screen.
- Local evaluation of no-RAG, fixed-RAG and agentic-RAG conditions.

12.4 SCALABILITY

The prototype targets one machine. Wider use needs queues, request isolation, retrieval monitoring, per-user authorization, protected secrets, rate limits, data rights and a stronger database design. More discovery candidates multiply inference and data cost. No throughput or capacity claim is made.

12.5 REAL-WORLD APPLICABILITY

Financial research use would also require broader sources, filings or full-article verification and expert review. Paper integration demonstrates an execution boundary; it does not authorize real investment execution. As a research prototype, AGENTTRADE demonstrates how multi-agent financial analysis can be combined with deterministic risk controls and human-approved paper execution. Real-world deployment would require broader financial data coverage, licensed sources, stronger security, operational monitoring, regulatory compliance and expert

validation., the system lets students compare retrieval policies and lets evaluators inspect a human-in-the-loop trading simulation.

13 CONCLUSION AND FUTURE WORK

13.1 CONCLUSION

AGENTTRADE implements bounded stock research, evidence retrieval, candidate discovery, Bull-Bear debate, Critic-based evaluation, bounded correction and human-approved paper execution. Executed verification confirmed the tested technical-indicator calculations, configured risk constraints, human-approval boundary, smoke-test functionality and end-to-end AAPL paper-trading workflow. The reported retrieval benchmark further shows that Fixed RAG achieved higher label-term accuracy than Agentic RAG for both evaluated 3B models. These results demonstrate workflow and implementation reliability for the tested scenarios, but do not establish predictive investment accuracy, profitability or production-grade security.

13.2 LIMITATIONS

- The benchmark corpus and question set are small and synthetic, limiting generalization.
- Label-term accuracy, citation syntax and reasoning proxies do not establish expert financial accuracy or source relevance.
- The evaluation does not measure predictive accuracy, profitability, retrieval precision/recall or trading performance.
- Raw benchmark answers and traces were not supplied, so the reported benchmark values were not independently rerun.
- Latency and computational cost were not measured.
- Plaintext credentials and incomplete authorization prevent safe public multi-user deployment.
- The current prototype does not establish production-grade security, regulatory compliance or real-money trading capability.

13.3 FUTURE WORK

- Strengthen behavioral and edge-case testing and preserve clear validation of model outputs.
- Preserve raw benchmark answers, traces and model versions, and add expert source-support evaluation.
- Evaluate 7B/3B model routing and discovery separately with measured latency and quality.
- Improve security through password hashing, protected secrets, session management and authorization controls.
- Reconcile workflow variants and replace UI-timed progress indicators with authoritative execution status.
- Expand evaluation to larger datasets, broader financial sources and more realistic research scenarios.

Local validation checklist. Use the intended implementation version, configure secrets locally without including them in source files, run docker compose config, then rebuild the API. Install the native models and publish one intended research workflow. Run the smoke suite and record every failure. Check a named stock, open-ended discovery, proposal rejection and expiry, and paper approval. These are required checks, not claimed passes.

REFERENCES

- [1] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401. <https://arxiv.org/abs/2005.11401>
- [2] Yao, S. et al. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629. <https://arxiv.org/abs/2210.03629>
- [3] n8n — n8n Docs. <https://docs.n8n.io/> (accessed 4 October 2026)
- [4] Ollama — Ollama Documentation. <https://docs.ollama.com/> (accessed 4 October 2026)
- [5] Qdrant — Documentation. <https://qdrant.tech/documentation/> (accessed 4 October 2026)
- [6] Alpaca — Paper Trading. <https://docs.alpaca.markets/docs/paper-trading> (accessed 4 October 2026)

